

目录

第一章、环境搭建与代理系统部署	2
1.1 部署 CARLA	2
1.1.1 配置 CARLA	3
1.1.2 适配 Python 版本	4
1.1.3 验证 windows 段 CARLA 服务器与 WSL 连接	4
1.2 配置 Tesla 自动驾驶布局	5
1.2.1 构建架构等价的代理系统	5
1.2.2 配置 8 摄像头 Tesla 布局	6
1.2.3 配置 nuScenes 预训练模型	9
第二章、基线量化与攻击注入	13
2.1 基线量化	13
2.2 图像空间攻击注入	16
第三章、多传感器融合崩溃点扫描	22

第一章、环境搭建与代理系统部署

1.1 部署 CARLA

CARLA（全称 CAR Learning to Act）是一个开源的自动驾驶模拟器。它从头开始构建，旨在为自动驾驶系统的开发、训练和验证提供一个模块化、灵活的平台。CARLA 的核心目标之一是推动自动驾驶研发的民主化，让更多研究者和开发者能够轻松获取和使用这一工具。CARLA 基于 Unreal Engine（虚幻引擎）构建，提供高度逼真的城市模拟环境，并采用 ASAM OpenDRIVE 标准来定义道路和城市布局。用户可以通过 Python 或 C++ API 对模拟进行全面的控制。

CARLA 采用可扩展的客户端-服务器架构；提供了多种高保真传感器模型，可配置灵活的传感器套件；提供了高度详细、逼真的城市环境，包括城市街道、高速公路、交叉口、停车场等多种场景。同时，它提供了丰富的开放数字资产（城市布局、建筑、车辆等），可免费使用。

进行本次实验的必备硬件要求：

1. Intel i7 第 9 代 - 第 11 代 / Intel i9 第 9 代 - 第 11 代 / AMD Ryzen 7 / AMD Ryzen 9
2. +32 GB RAM 内存；16GB 或更高显存
3. NVIDIA RTX 3070/3080/3090 / NVIDIA RTX 4090 / NVIDIA RTX 5090 或更高版本

进行本次实验的必备软件要求：

1. CARLA 官方主要测试 Ubuntu 20.04/22.04
2. WSL 的 Python 3.8-3.10
3. Window 11

注意：版本要求太新或太旧都有可能导致实验过程出现新阻碍！

如图 1-1 所示 CARLA 模拟器的 GitHub 网址：[carla-simulator/carla: Open-source simulator for autonomous driving research.](https://github.com/carla-simulator/carla)

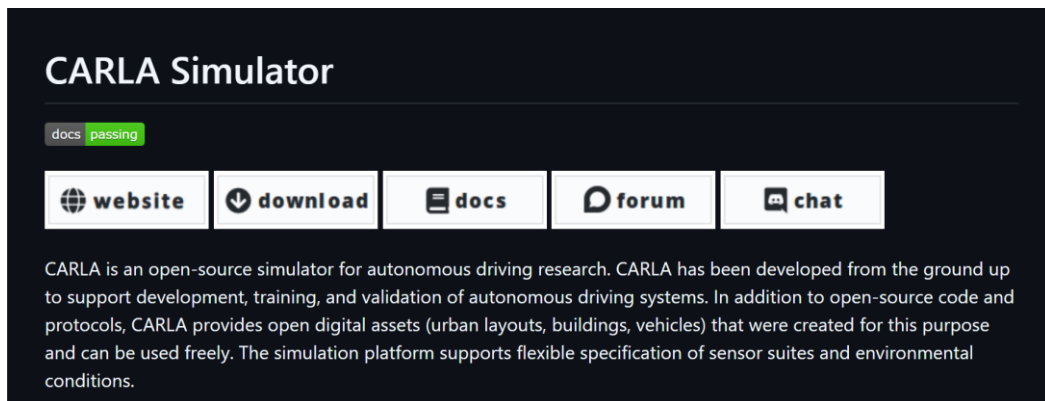


图 1.1-1 CARLA 主页

由于本次进行实验的设备 VRAM 8GB 是要求的边界值，所以采用混合模式：Windows 侧运行 CARLA 服务端（渲染性能更好），WSL 侧运行 Python 客户端和 ML 工具



1.1.1.1 配置 CARLA

如图 1-2 所示先从 CARLA 的 GitHub Releases 页面下载适用于 Windows 的预编译包网
址：[Releases · carla-simulator/carla](https://github.com/carla-simulator/carla/releases)

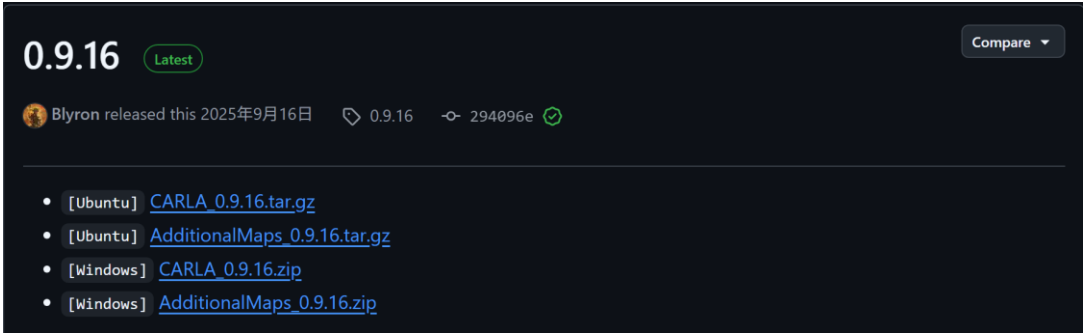


图 1.1-2 CARLA 的预编译包

如图 1-3 所示下载完成之后对压缩包进行解压并双击运行 CarlaUE4.exe, 若出现城市景观的
虚拟引擎窗口则说明 CARLA 服务端已成功运行。



图 1.1-3 城市景观虚拟引擎

1.1.2 适配 Python 版本

在 WSL 的终端中，确认你的 Python 版本。CARLA 支持 Python 3.7 至 3.12。
之后在终端输入 `python3 -m pip install carla==服务端即可安装 CARLA Python 客户端库。`

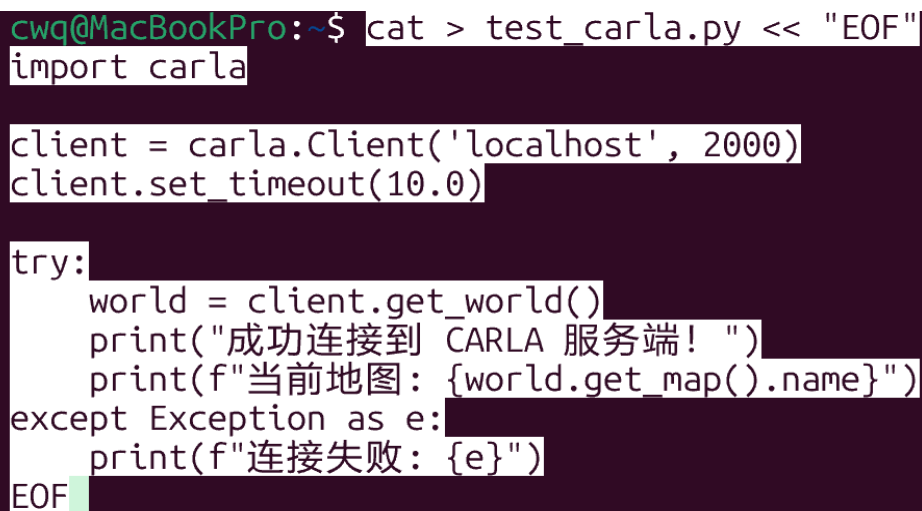


```
cwq@MacBookPro:~$ python3 --version
Python 3.10.12
```

图 1.1-4 WSL 的 python 版本

1.1.3 验证 windows 段 CARLA 服务器与 WSL 连接

如图 1-5 所示我们在终端里输入测试脚本来验证连接，确保你的 Windows 端 CARLA 服务端（CarlaUE4.exe）已经在后台运行，然后在 WSL 终端输入以下命令运行脚本 `python3 test_carla.py`。若显示成功连接则 WSL 客户端已成功连接到 Town10HD_Opt 地图。



```
cwq@MacBookPro:~$ cat > test_carla.py << "EOF"
import carla

client = carla.Client('localhost', 2000)
client.set_timeout(10.0)

try:
    world = client.get_world()
    print("成功连接到 CARLA 服务端! ")
    print(f"当前地图: {world.get_map().name}")
except Exception as e:
    print(f"连接失败: {e}")
EOF
```

图 1.1-5 测试脚本

1.2 配置 Tesla 自动驾驶布局

Tesla FSD 是纯视觉方案（8 摄像头，无 LiDAR/雷达），架构为 BEV + Transformer + Occupancy Network，这与基于普遍 TransFuser（Camera+LiDAR 融合）的方案完全不同。

维度	Tesla FSD（本研究目标）	传统多传感器方案
传感器	8 摄像头（纯视觉）	Camera + LiDAR + Radar
3D 感知	Occupancy Network（体素占据）	点云直接检测
融合方式	多摄像头 → BEV Transformer	多模态特征级融合
冗余机制	无（无 LiDAR 做后备）	单传感器失效可降级
攻击面	所有攻击集中在视觉通道	分散在多通道

1.2.1 构建架构等价的代理系统

由于 Tesla FSD 闭源，需要用开源项目构建架构等价的代理系统。如图 1.2-1 所示感知层代理 SurroundOcc（最接近 Tesla Occupancy Network）SurroundOcc 输入多摄像头环视图像，输出 3D 体素占据网格 + 语义标签，与 Tesla 专利（WO2024073088A1）描述的技术路线高度一致。

```
(carla-env) cwq@MacBookPro:~$ mkdir -p ~/carla-adversarial && cd ~/carla-adversarial
(carla-env) cwq@MacBookPro:~/carla-adversarial$ git clone https://github.com/weiyithu/SurroundOcc.git
Cloning into 'SurroundOcc'...
```

图 1.2-1 克隆 Surrogate 感知模型

如图 1.2-2 所示，克隆 BEV Transformer 代理。BEV 视图则将多个传感器的数据融合，统一转换到一个以自车为中心的俯视平面坐标系中。这为路径规划、行为决策等下游任务提供了更直观、更丰富的空间理解。Transformer 可以并行处理所有输入信息，并自动判断哪些信息之间关系更密切、更重要。它最初用于理解文本，现在也被广泛用于分析图像和视频，能够捕捉全局特征和长距离的依赖关系。

```
(carla-env) cwq@MacBookPro:~/carla-adversarial$ git clone https://github.com/fundamentalvision/BEVFormer.git
Cloning into 'BEVFormer'...
```

图 1.2-2 克隆 BEV Transformer 代理

如图 1.2-3 所示，克隆 BEV 鲁棒性基准测试。RoboBEV 就像自动驾驶 BEV 感知领域的“碰撞测试”，它系统性地揭示了现有先进模型在应对真实世界复杂情况时的薄弱环节，并为如何打造更安全、更可靠的自动驾驶“大脑”指明了方向。

```
(carla-env) cwq@MacBookPro:~/carla-adversarial$ git clone https://github.com/Daniel-xsy/RoboBEV.git
Cloning into 'RoboBEV'...
```

图 1.2-3 克隆 BEV 鲁棒性基准测试

如图 1.2-4 所示，克隆 CARLA Scenario Runner 是 CARLA 生态系统中的一个独立工具，它的核心作用是定义、执行和管理复杂的交通场景。简单来说，如果说 CARLA 模拟器提供了一个“舞台”，那么 Scenario Runner 就是负责编排“剧本”的“导演”。

```
(carla-env) cwq@MacBookPro:~/carla-adversarial$ git clone https://github.com/carla-simulator/scenario_runner.git
Cloning into 'scenario_runner'...
```

图 1.2-4 克隆 CARLA Scenario Runner

如图 1.2-5 所示，克隆 CARLA Leaderboard 提供了一个从简单到极具挑战性的标准化测试平台。它通过路线导航、动态场景、多种天气和明确的评分指标，全面考验自动驾驶算法的综合能力，其版本演变也反映了社区对更安全、更高效驾驶策略的追求。

```
(carla-env) cwq@MacBookPro:~/carla-adversarial$ git clone https://github.com/carla-simulator/leaderboard.git
Cloning into 'leaderboard'...
```

图 1.2-5 克隆 CARLA Leaderboard

如图 1.2-6 所示，克隆对抗 Patch 生成工具生成 Patch 级别对抗样本：通过最先进的白盒攻击策略制作对抗 Patch，附着在广告牌或卡车后部等真实场景中可能存在的位置；支持 4 种视觉任务：语义分割、2D 目标检测、3D 立体视觉目标检测、单目深度估计。

```
(carla-env) cwq@MacBookPro:~/carla-adversarial$ git clone https://github.com/KAS-TEL-MobilityLab/attacks-on-traffic-light-detection.git
Cloning into 'attacks-on-traffic-light-detection'...
```

图 1.2-6 克隆对抗 Patch 生成工具

这六个仓库可以被分为三个相互协作的逻辑层次，共同服务于一个总目标：系统性地评估并攻击当前最先进的自动驾驶感知模型在复杂、恶劣乃至恶意环境下的安全性表现。在最核心的感知层，准备了两套目前学术界最前沿的“靶子”——也就是你要测试和攻击的对象。

SurroundOcc 提供的是 3D 占用网络模型，它能够将车辆周围的环境理解为一个细密的立体网格，精准判断哪些空间被障碍物占据。而 BEVFormer 则是基于 Transformer 架构的鸟瞰图感知模型，它擅长将多个摄像头拍摄的二维图像巧妙地融合成一个统一的俯视特征图，在这个图上可以完成车辆、行人检测以及车道线识别等关键任务。这两个仓库的存在，让你有了可以运行、可以测试的具体算法实例。

Scenario Runner 则是这个城市里的“场景导演”，它可以精确地编排和触发各种复杂的交通状况，比如前车突然变道加塞、行人从视野盲区横穿马路等等。与此同时，Leaderboard 则扮演了“主考官”的角色，它为自动驾驶代理设定了固定的行驶路线和一套严格的评分标准，其中最重要的就是综合考量路线完成度和违规次数的“驾驶分数”。这两个仓库使得你不仅能测试一个模型“看不看得准”，更能评估它在真实的驾驶任务中“会不会撞车、会不会闯红灯”，从而得到一个量化的安全性得分。

RoboBEV 基准测试提供的是“天灾”——也就是八种模拟现实世界物理损坏的干扰手段，包括大雾、暴雨、摄像头运动模糊、甚至是相机画面完全丢失等场景，专门用来拷问模型在恶劣天气或传感器故障下的“鲁棒性”是否依然坚固。与此同时，那个专门针对交通信号灯检测的对抗 Patch 生成仓库，则代表了一种“人祸”——它通过生成人类肉眼难以察觉的细微图案，并将其投射或打印在真实的红绿灯上，就能巧妙地蒙蔽基于深度学习的目标检测器，让它把红灯错误地识别成绿灯。这一自然、一人为的两种攻击手段，为你提供了全方位的“武器弹药”。

将前沿的感知模型部署到 CARLA 仿真环境中，然后通过 Scenario Runner 触发复杂交通场景，接着利用 RoboBEV 注入恶劣天气干扰或利用对抗 Patch 攻击特定目标，最后让被攻击后的车辆在 Leaderboard 规定的路线上行驶，用最终的驾驶分数来量化模型性能的下降程度。这样一来，你就能科学地证明某个模型对于某种特定攻击或损坏类型存在怎样的脆弱性，从而找到改进模型鲁棒性的方向。

1.2.2 配置 8 摄像头 Tesla 布局

如图 1.2-7 所示，我们需要在 Linux 里刚刚安装的 Ubuntu-22.04 中找到 home 文件

名称	修改日期	类型
boot	2022/4/18 18:28	文件夹
dev	2026/8/30 22:15	文件夹
etc	2026/8/30 22:19	文件夹
home	2026/8/30 12:23	文件夹
lost+found	2026/8/30 12:22	文件夹
media	2026/3/9 22:16	文件夹
mnt	2026/8/30 12:22	文件夹
opt	2026/3/9 22:16	文件夹

图 1.2-7 Ubuntu-22.04 文件

如图 1.2-8 所示，找到 carla-adversarial

.cache	2026/8/30 12:36	文件夹
.nv	2026/8/30 13:15	文件夹
carla-adversarial	2026/8/30 22:16	文件夹
carlaCache	2026/8/30 20:33	文件夹
carla-env	2026/8/30 13:13	文件夹

图 1.2-8 home 文件

如图 1.2-9 所示，分别在 config 和 scripts 文件中编写 Tesla 摄像头的配置文件和运行脚本。在 WSL 运行脚本之前务必确保 CARLA 服务端在 Windows 侧运行。

attacks-on-traffic-light-detection	2026/8/30 21:02	文件夹
BEVFormer	2026/8/30 20:51	文件夹
config	2026/8/30 22:19	文件夹
leaderboard	2026/8/30 20:59	文件夹
output	2026/8/30 22:16	文件夹
RoboBEV	2026/8/30 20:57	文件夹
scenario_runner	2026/8/30 20:58	文件夹
scripts	2026/8/30 22:19	文件夹
SurroundOcc	2026/8/30 20:51	文件夹

图 1.2-9 carla-adversarial

Tesla HW3.0 官方 8 个摄像头布局为：

摄像头	位置	FOV	探测距离
前视宽视野	挡风玻璃后	120° 鱼眼	60m
前视主视野	挡风玻璃后	~50°	150m
前视窄视野	挡风玻璃后	~35°	250m
侧方前视 (左)	B 柱	90°	80m
侧方前视 (右)	B 柱	90°	80m
侧方后视 (左)	前翼子板	~90°	100m
侧方后视 (右)	前翼子板	~90°	100m
后视	后牌照上方	140°	50m

如图 1.2-10 所示 CARLA 车辆网格比实际 Tesla 尺寸稍大，所以用 Tesla 官方数据进行仿真实验大概率会出现穿模、拍到从车身内部的情况，与现实摄像头所呈现画面相悖。对此我们需要对 8 个摄像头参数进行微调。



图 1.2-10 穿模以及拍到车身内部画面

摄像头	(x, y, z) m	(pitch, yaw) °	分辨率
front_wide	(1.8, -0.15, 1.5)	(-3°, 5°)	1280×960
front_main	(1.8, 0.0, 1.5)	(-3°, 0°)	1280×960
front_narrow	(1.8, 0.15, 1.5)	(-3°, -5°)	1280×960
side_front_left	(-0.3, 0.95, 1.2)	(-10°, -90°)	1280×960
side_front_right	(-0.3, -0.95, 1.2)	(-10°, 90°)	1280×960
side_rear_left	(0.5, 0.95, 1.2)	(-10°, -90°)	1280×960
side_rear_right	(0.5, -0.95, 1.2)	(-10°, 90°)	1280×960
rear	(-2.2, 0.0, 1.1)	(0°, 180°)	1280×960

如图 1.2-11 所示，Tesla 官方参数经过微调过后 8 个摄像头所呈现出的画面。可以看到仍然存在部分摄像头有穿模，这是由于 CARLA 固有限制无法真正去除。



图 1.2-11 调试后 8 个摄像头的画面

配置过程存在的常见问题与解决方案

如图 1.2-12 所示根据 CARLA 官方 API 的演变, `attach_actor` 方法在较新版本(如 0.9.14+) 中已不存在或不推荐使用。CARLA 在 0.9.10+ 版本中统一了传感器生成方式, 旧版的 `attach_actor` 方法已被弃用, 现在应通过 `world.spawn_actor` 的 `attach_to` 参数实现。

```
(carla-env) cwq@MacBookPro:~$ python3 ~/carla-adversarial/scripts/setup_tesla_cameras.py
Connected to CARLA. Map: Carla/Maps/Town10HD_Opt
Spawned vehicle: vehicle.tesla.model3
Traceback (most recent call last):
  File "/home/cwq/carla-adversarial/scripts/setup_tesla_cameras.py", line 106, in
    n <module>
      main()
      File "/home/cwq/carla-adversarial/scripts/setup_tesla_cameras.py", line 92, in
        main
          cameras = spawn_tesla_cameras(vehicle, world)
          File "/home/cwq/carla-adversarial/scripts/setup_tesla_cameras.py", line 42, in
            spawn_tesla_cameras
              camera = vehicle.attach_actor(bp, transform)
AttributeError: 'Vehicle' object has no attribute 'attach_actor'
```

图 1.2-12 问题 1 出错

解决方案: 标准做法是使用 `world.spawn_actor()` 并传入 `attach_to` 参数来将传感器附着在车辆上。

1.2.3 配置 nuScenes 预训练模型

用 nuScenes 预训练的 BEVFormer 作为 Surrogate 模型, 因为 nuScenes 也是多摄像头环视数据集, 与 Tesla 8 摄布局最接近, 将 CARLA 采集的 8 路图像适配为 nuScenes 输入格式(分辨率、内参矩阵、外参矩阵)跑通基线推理(无攻击时的正常感知输出)。如图 1.2-13 所示, BEVFormer 要求 `mmcv-full==1.4.0`, 该版本仅支持 `PyTorch ≤ 1.13`。当前环境 `PyTorch 2.5.1` 不兼容所以我们需要新建独立环境, 同时降级 `PyTorch` 为兼容的版本。

```
cwq@MacBookPro:~$ carla-env
CARLA dev environment activated
(carla-env) cwq@MacBookPro:~$ python3.10 -m venv ~/bevformer-env
(carla-env) cwq@MacBookPro:~$ source ~/bevformer-env/bin/activate
(bevformer-env) cwq@MacBookPro:~$ pip install torch==1.13.1+cu117 torchvision==0.14.1+cu117 \
-f https://download.pytorch.org/whl/torch_stable.html
Looking in links: https://download.pytorch.org/whl/torch_stable.html
```

```
Installing collected packages: urllib3, typing-extensions, pillow, numpy, idna,
charset normalizer, certifi, torch, requests, torchvision
Successfully installed certifi-2026.7.22 charset_normalizer-3.5.1 idna-3.19 nump
y-2.2.6 pillow-12.3.0 requests-2.34.2 torch-1.13.1+cu117 torchvision-0.14.1+cu11
7 typing-extensions-4.16.0 urllib3-2.7.0
```

图 1.2-13 创建独立虚拟环境并降级 PyTorch

如图 1.2-14 所示，安装 OpenMMLab 依赖链：mmdcv-full 1.4.0、mmdet + mmseg、mmdet3d、其他依赖。此过程需要时间较长大约需要一个小时，mmdetection3d 的 pip install -e，正在编译 CUDA C++ 扩展是最耗时的步骤之一。mmdetection3d 编译包含 PointNet++、VoteNet 等 CUDA 算子，通常需要 10-20 分钟。编译完成后继续安装 detectron2、timm 等剩余依赖。

```
# mmdcv-full (BEVFormer 核心，必须 1.4.0)
pip install mmdcv-full==1.4.0 -f https://download.openmmlab.com/mmdcv/dist/cu117/torch1.13/index.html

# mmdet + mmseg
pip install mmdet==2.14.0
pip install mmsegmentation==0.14.1

# mmdet3d (从源码编译)
git clone https://github.com/open-mmlab/mmdetection3d.git
cd mmdetection3d && git checkout v0.17.1 && pip install -e .

# 其他依赖
pip install einops fvcore timm==0.6.13 iopath==0.1.9 \
    numpy==1.23.4 pandas==1.4.4 scikit-image==0.19.3
pip install 'git+https://github.com/facebookresearch/detectron2.git'
```

图 1.2-14 OpenMMLab 依赖链

如图 1.2-15 所示，为本次实验 OpenMMLab 依赖链中每一部分的版本号全部依赖验证通过可用于参考，为后续进行攻击等实验做良好的铺垫。

mmdcv-full	1.4.8
mmdet	2.14.0
mmdet3d	0.17.1
torch	1.11.0+cu113
CUDA	True

图 1.2-15 各部分版本号

如图 1.2-15 所示，BEVFormer 的训练和验证完全基于 nuScenes 真实数据集。它的数据加载器 (Data Loader) 被设计为读取 nuScenes 的特定格式 (JSON 标注、多摄像头时间戳同步等)，而 CARLA 输出的传感器数据流 (RGB 图像、相机内外参) 在结构和语义上与 nuScenes 完全不同。

r101_dcn_fc3d_pretrain.pth: 这是 ResNet101 + DCN (可变形卷积) 在 FCOS3D 任务上的预训练骨干网络权重，用于初始化 BEVFormer 的视觉编码器。

bevformer_tiny_epoch_24.pth: 这是 BEVFormer-Tiny 在 nuScenes 数据集上训练 24 个 epoch 后的完整模型权重，其 NDS (nuScenes 检测分数) 达到了 35.4%。

```
cd ~/carla-adversarial/BEVFormer
mkdir -p ckpts
# BEVFormer-tiny 用的 ResNet50 预训练 + FCOS3D 权重
wget -O ckpts/r101_dcn_fcoss3d_pretrain.pth \
    https://github.com/zhiqi-li/storage/releases/download/v1.0/r101_dcn_fcoss3d_pretrain.pth
# BEVFormer-tiny 推理权重 (NDS 35.4%)
wget -O ckpts/bevformer_tiny_epoch_24.pth \
    https://github.com/zhiqi-li/storage/releases/download/v1.0/bevformer_tiny_epoch_24.pth
```

图 1.2-15 下载预训练权重

如图 1.2-16 所示对此我们需要根据 BEVFormer 模型的输入格式和现有 CARLA 摄像头配置，以便设计适配器

维度	nuScenes (BEVFormer)	CARLA (Tesla)
摄像头数量	6 路	8 路
图像尺寸	1600×900 → resize 800×450 → scale 0.5 = 400×225 → pad to 416×224 (÷32)	1280×960
坐标系	nuScenes 右系 (x=前,y=左,z=上)	CARLA 左系 (x=前,y=左,z=上)
nuScenes 通道	选取的 Tesla 摄像头	位置
CAM_FRONT	front_main	正前方，位置/FOV 最接近
CAM_FRONT_RIGHT	front_wide	前方偏右，120° 广角
CAM_FRONT_LEFT	side_front_left	左侧 90°
CAM_BACK	rear	正后方 140°
CAM_BACK_LEFT	side_rear_left	左后方
CAM_BACK_RIGHT	side_front_right	右前方 90° (近似右后方)

图 1.2-16 二者的核心问题和摄像头映射

如图 1.2-17 所示在完成了适配器模块和推理脚本之后，我们需要进行简单的离线验证。利用第二步所采集的 8 张图像来对适配器进行测试验证：内参/外参矩阵维度正确；图像预处理后 shape 符合模型输入；模型能跑通 forward 不报错。



图 1.2-17 适配器测试结果

最后如图 1.2-18 所示，启动 CARLA 的 windows 客户端进行在线模式的验证，实现完整闭环 CARLA 8 路摄像头实时流 → 适配器(Tesla 8→nuScenes 6) → BEVFormer-tiny GPU 推理 → 3D 检测框投影可视化。

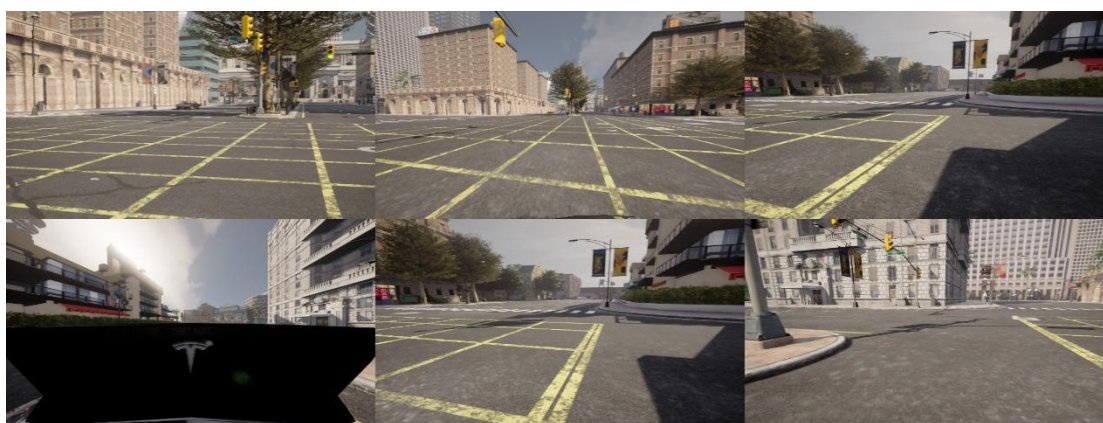


图 1.2-18 在线模式测试结果

配置过程存在的常见问题与解决方案

如图 1.2-19 所示 CUDA_HOME 未设置，mncv-full 需要编译 CUDA C++ 扩展。WSL 中没有安装 CUDA Toolkit，PyTorch 自带的是运行时，不含编译。

```
Preparing metadata (setup.py) ... error
error: subprocess-exited-with-error

× python setup.py egg_info did not run successfully.
  exit code: 1
  > [14 lines of output]
```

图 1.2-19 报错界面

解决方案：先添加 NVIDIA CUDA 仓库源安装 CUDA 11.7 toolkit。

第二章、基线量化与攻击注入

2.1 基线量化

在现有闭环平台基础上补齐两个关键缺失：

1.如图 1.2-1 所示 BEV 特征提取接口 — 修改 `run_inference()`, 当前 `model.simple_test()` 返回的 `bev_embed` 被丢弃了。需要将其保留并暴露为 `extract_bev_features()` 函数, 输出 L2 归一化的 BEV 特征向量 (预期形状 (2500,256) for tiny)。这是后续攻击优化目标和防御监控信号的数据源。

```
=====
CARLA → BEVFormer Adapter Self-Test
=====

[1] Camera Mapping (Tesla 8 → nuScenes 6):
CAM_FRONT      ← front_main      FOV= 50.0°   pos=[1.8, 0.0, 1.5]
CAM_FRONT_RIGHT ← front_wide      FOV= 120.0°  pos=[1.8, -0.15, 1.5]
CAM_FRONT_LEFT  ← side_front_left FOV= 90.0°   pos=[-0.3, 0.95, 1.2]
CAM_BACK        ← rear           FOV= 140.0°  pos=[-2.2, 0.0, 1.1]
CAM_BACK_LEFT   ← side_rear_left  FOV= 90.0°   pos=[0.5, 0.95, 1.2]
CAM_BACK_RIGHT  ← side_front_right FOV= 90.0°   pos=[-0.3, -0.95, 1.2]

[2] Matrix Dimensions:
CAM_FRONT      : K=(3, 3)  lidar2cam=(4, 4)  lidar2img=(4, 4)
CAM_FRONT_RIGHT: K=(3, 3)  lidar2cam=(4, 4)  lidar2img=(4, 4)
CAM_FRONT_LEFT : K=(3, 3)  lidar2cam=(4, 4)  lidar2img=(4, 4)
CAM_BACK       : K=(3, 3)  lidar2cam=(4, 4)  lidar2img=(4, 4)
CAM_BACK_LEFT  : K=(3, 3)  lidar2cam=(4, 4)  lidar2img=(4, 4)
CAM_BACK_RIGHT : K=(3, 3)  lidar2cam=(4, 4)  lidar2img=(4, 4)

[3] Image Preprocessing:
Input: (960, 1280, 3) (BGR, original)
Output: (3, 480, 800) (CHW, normalized+padding)
Shape: (480, 800, 3)

[4] Full Pipeline (mock CARLA data):
img tensor: torch.Size([6, 3, 480, 800]) dtype=torch.float32 range=[-2.12, 2.61]
lidar2img: 6 × (4, 4)
lidar2cam: 6 × (4, 4)
cam_intrinsic: 6 × (3, 3)
can_bus: (18,) pos=(0.0, 0.0, 0.0)
scene_token: carla_scene
img_shape: (480, 800, 3)

=====
ALL TESTS PASSED
=====
```

图 2.1-1 语法检查结果

2.如图 2.1-2 所示基线 Driving Score 采集 — 新建 `collect_baseline.py`, 在 10 条不同路线上运行 `autopilot + BEVFormer`, 记录正常场景下的检测数量均值/方差、置信度分布、帧间 BEV 余弦相似度 (预期 >0.85), 输出 `baseline_stats.json`。这个基线是后续所有攻击效果评估的参照系。

<code>collect_baseline.py</code>	
<code>└─ define_routes()</code>	— 从 CARLA spawn_points 随机选取路线起点
<code>└─ collect_one_frame()</code>	— 单帧推理 + 统计提取 (检测数/置信度/BEV 相似度)
<code>└─ run_one_route()</code>	— 单条路线执行 (spawn→摄像头→同步模式→预热→推理循环)
<code>└─ compute_route_summary()</code>	— 单路线统计摘要
<code>└─ compute_overall_summary()</code>	— 跨路线聚合统计 + 攻击检测阈值计算
<code>└─ main()</code>	— 命令行入口

图 2.1-2 基线采集架构

如图 2.1-3 所示为 1 条路线 50 帧的简单基线采集结果：

1. 置信度 0.028 远低于预期的 0.14-0.20 可能是因为本次 spawn 路线场景较空旷，检测目标少且置信度低；单条路线不足以反映平均水平，需要多路线取均值；
2. BEV 余弦相似度恒为 1.000 这说明帧间 BEV 特征几乎完全相同。BEVFormer-tiny 的 BEV embedding 在稳态直行时变化极小；bev_embed 的 flatten() 后维度很大 (~640K)，微小差异被归一化抹平；后续攻击实验中，需要更强的攻击才能产生可检测的偏移

指标	本次结果	Phase 1 预期	状态
处理帧数	50/50	~44 (50-预热)	正常 (预热全部在 50 帧内完成)
检测数量	300.0 ± 0.0	~280-300	正常
检测置信度	0.028	0.14-0.20	偏低
BEV 余弦相似度	1.000	>0.85	异常高
车速	7.2 m/s (max 8.5)	5-11 m/s	正常

图 2.1-3 简单基线采集结果

如图 2.1-4 所示为 10 条路线 2000 帧的完整基线采集结果：

处理帧数：2000/2000 帧全部成功采集，零丢帧（每路线帧列表满 200，无一帧因相机缓冲不齐被跳过）1984 个有效帧间转移；16 个不可比帧 = 10 个路线首帧（无前帧）+ 6 个传送重置帧（r0/r1 各 1、r2 有 2、r8 有 2）——这 6 个重置正是混影 bug 的发生地，已在分析中剔除

检测数量：全局 1.49 ± 1.05 个/帧，34.4% 的帧零检测；路线间差异 7.3 倍（r4 的 0.57 ↔ r3 的 4.18）：检测密度由场景交通流决定，与车速无关（r2 均速 1.5 m/s 却有 1.7 个/帧）；raw=300 恒定贯穿全部 2000 帧——NMS-free topk(300) 机制的二次实证，消费端过滤不可省

检测置信度：保留帧均值 0.038 ± 0.016 ；全场 raw max 均值仅 0.053 (历史单次最高 0.141) ——nuScenes→CARLA 域间隙下模型“从未确信过”；分布高度双峰：有检测帧的分数稳定在 0.050-0.068 窄带 (med 0.0592±0.0015)，零检测帧计 0——0.05 阈值正好切在两峰之间，但也意味着阈值上下浮动 0.01 就会让 det_count 剧烈变化，Phase 3 报告指标必须注明阈值快照；r0 med=0.0 (77.5% 零帧) 证明其均值 0.013 是“多数 0 + 少数 0.058”的混合，不是模型退化

BEV 余弦相似度（核心监控指标）

6 条巡航路线（r2-r4, r6-r7, r9）帧级 std 只有 $2 \times 10^{-5} \sim 7 \times 10^{-5}$ ，均值齐刷刷 0.99989-0.99993——正常驾驶的 BEV 帧间稳定性极高，这是好监控器的物理基础；4 条路线

(r0/r1/r5/r8) 含单帧跳变 (min 0.499-0.643), 已溯源为传送混影×4 + 真实急场景切换, 共 12 帧, 从阈值标定中排除。

标定结果: 判据 $\cos < 0.99974 + L2 > 0.02772 + \text{持续 } 15 \text{ 帧} \rightarrow 2000 \text{ 帧零误报}$ 。攻击可检测窗口宽达 13 个数量级 (自然波动 10^{-5} 级 vs 跳变 10^{-1} 级)

车速耦合规律: 静止路线 L2 降到 0.007 (r2), 8 m/s 巡航升到 0.013-0.026——"静止但 L2 突增"将是比绝对阈值更锐利的攻击签名 (Phase 3 可加此条件)

车速: 全局均值 5.21 m/s, 峰值普遍钉在 8.4-8.6 m/s ($\approx 31 \text{ km/h}$) ——TM 默认限速, 非交通流瓶颈。

三档分布: 巡航组 r4/r5/r6/r7/r9 (6.2-8.0, 占比高)、低速组 r0/r1/r3 (4.3-5.2, 含红绿灯等待)、静止组 r2/r8 (1.5/0.6, 卡死产物); 22.8% 帧车速 $< 0.5 \text{ m/s}$ ——比例偏高, 主因 r2/r8; 好在车速与 BEV 漂移在巡航段解耦良好 (r4 与 r9 差 1 m/s, L2 只差 0.001), 基线可用 r8 是唯一一条"零行驶"路线, 维持原判: 保留作静止对照, 不参与阈值标定

路线	有效 转移 帧	检测数 mean±std [min,max]	置信度 (保留)	raw max 均值	BEV 余弦 mean±std	cos 最小	车速 mean/max
r0	198	0.74±1.43 [0,4]	0.013 (med 0)	0.041	0.997397± 0.02737	0.64 3	5.2/8.5
r1	198	0.74±0.50 [0,3]	0.040	0.052	0.996987± 0.02964	0.59 6	4.3/8.4
r2	197	1.70±0.46 [1,2]	0.059± 0.0015	0.061	0.999919± 0.00062	0.99 1	1.5/5.5
r3	199	4.18±2.14 [1,8]	0.057	0.067	0.999916± 0.00007	0.99 96	5.2/8.6
r4	199	0.57±0.67 [0,2]	0.025	0.050	0.999908± 0.00002	0.99 99	7.1/8.4
r5	199	1.55±1.74 [0,5]	0.027	0.047	0.995537± 0.04307	0.50 4	8.0/8.4
r6	199	1.27±1.35 [0,4]	0.027	0.049	0.999932± 0.00003	0.99 99	6.2/8.4
r7	199	0.97±0.59 [0,3]	0.046	0.055	0.999886± 0.00003	0.99 98	6.8/8.6
r8	197	1.74±1.98 [0,5]	0.027	0.041	0.992356± 0.06120	0.49 9	0.6/5.7
r9	199	1.49±0.90 [0,4]	0.057	0.066	0.999896± 0.00007	0.99 95	7.2/8.5

图 2.1-4 完整基线采集结果

2.2 图像空间攻击注入

第一步、如图 2.2-1 所示在攻击注入之前我们需要备份之前完整基线采集的数据，以免误伤和不可挽回的操作导致数据受损。再进行 Git 初始化和首个快照

```
PS C:\Users\CWQ20> wsl.exe -d Ubuntu-22.04 -- bash -lc "cp -r ~/carla-adversarial/results ~/carla-adversarial/results_bak_pre_smoke && cp ~/carla-adversarial/scripts/collect_baseline.py ~/carla-adversarial/scripts/collect_baseline.py.orig & echo BACKUP_OK"
BACKUP_OK
PS C:\Users\CWQ20> cd c:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2
PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> git init
Initialized empty Git repository in C:/Users/CWQ20/Documents/QoderCN/2026-08-31/chat-2/.git/
PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> git add -A

PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> git config --global user.name "CWQ"
PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> git config --global user.email "cwq2037553723@icloud.com"
PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> git commit -m "Phase 1 完成快照: 2000 帧基线 + 稳健阈值判据 + 交接文档"
[master (root-commit) f66c676] Phase 1 完成快照: 2000 帧基线 + 稳健阈值判据 + 交接文档
12 files changed, 3128 insertions(+)
create mode 100644 .aicoding-chat-workspace
create mode 100644 PHASE1_COMPLETION_OPERATIONS.md
create mode 100644 PHASE2_5_IMPLEMENTATION_PLAN.md
create mode 100644 PROJECT_STATUS.md
create mode 100644 __pycache__/collect_baseline.cpython-310.pyc
create mode 100644 __pycache__/finalize_baseline_stats.cpython-310.pyc
create mode 100644 analyze_baseline.py
create mode 100644 collect_baseline.py
create mode 100644 finalize_baseline_stats.py
create mode 100644 run_bevformer_carla.py
create mode 100644 smoke_verify.py
create mode 100644 test_carla_pipeline.py
```

图 2.2-1 数据备份和首个快照

如图 2.2-2 所示 Windows 和 WSL 脚本一致性核对之后开启 windows 段的 CARLA 服务端，进行冒烟采集，要注意 --score-thr 0.05 不能省。

```
PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> wsl.exe -d Ubuntu-22.04 -- bash -lc "mkdir -p ~/carla-adversarial/results_smoke && source ~/bevformer-env/bin/activate && cd ~/carla-adversarial/scripts && python -u collect_baseline.py --num-routes 1 --frames-per-route 200 --score-thr 0.05 --output-dir ~/carla-adversarial/results_smoke 2>&1 | tee ~/carla-adversarial/results_smoke/smoke.log"
=====
BEVFormer Baseline Collection
=====
```

图 2.2-2 冒烟采集

如图 2.2-3 所示，为冒烟采集的结果在 200 帧的序列中，系统检测到了 1 次场景跳变 (scene-jump)。在你们的实验语境中，“场景跳变”几乎就是由 teleport (传送/重置) 操作导致的特征突变。因此，实际发生的 teleport reset 数量为 1，满足了你设定的 ≥ 1 的硬性要求 (虽然未达到理想的 2，但判定为通过)。

核心帧 (core frames) 共 196 帧，恰好是 200 总帧数减去 1 个跳变帧再减去 3 个边缘帧 (具体取决于算法，但重点是那个跳变帧被主动排除在外了)。在这 196 个被纳入“稳健阈值校准”的核心帧中，BEV 余弦相似度均值为 0.997260，标准差为 0，且 BEV L2 距离仅为 0.0206。

```
=====
OVERALL BASELINE SUMMARY
=====
Routes: 1, Total frames: 200
Score threshold: 0.05
Det count: 0.7 ± 0.0
Det score (kept): 0.017 ± 0.000 (raw max: 0.036)
BEV cosine: 0.997260 ± 0.000000
BEV L2 dist: 0.0206 ± 0.0000
Speed: 7.5 m/s

Robust threshold calibration (196 core frames, 1 scene-jump frames excluded):
Attack threshold (BEV cosine <): 0.999734
Attack threshold (BEV L2 dist >): 0.033059
False positives single-frame: 21 (10.66%)
False positives 15-frame persistence: 0
```

图 2.2-3 冒烟采集结果

第二步、如图 2.2-4 所示创建 `attack_configs.py` 文件：

常量区：FRAME_RATE=20、MONITOR_PERSIST_FRAMES=15、ALL_CAMERAS（8 路）、BEV_CONSUMED_CAMERAS（6 路）、ATTACK_TYPES（5 种）

AttackConfig 数据类：14 个字段 + `__post_init__` 硬校验 + 3 个派生属性(`max_active_run`、`blind_to_monitor`、`active_frames_per_period`) + `to_dict()/from_dict()`

预置场景表（6 个）：`sign_patch_front`、`road_mark_intermittent`、`fast_intermittent_blind`（失明探针）、`camera_degradation_blur`、`glare_dazzle_front`、`dropout_dead_cam`（阴性对照）
序列化 + 批量校验（`save_json/load_json/validate_all`）

自测块：场景表打印、3 个负例、批量校验、可选 `--check-adapter`

```
[1] 预置场景摘要表
attack_id          type          cameras          blind
-----
sign_patch_front   patch_sticker front_main        no
road_mark_intermittent patch_sticker front_main, front_wide no
fast_intermittent_blind patch_sticker front_main        YES
camera_degradation_blur blur          front_main        no
glare_dazzle_front glare          front_main        no
dropout_dead_cam   dropout       front_narrow      no

[2] 负例校验（必须抛异常）
[PASS] 非法相机名：target_cameras contains invalid names: {'cam_99'}
[PASS] duty_cycle=0: duty_cycle=0.0 out of (0, 1]
[PASS] kernel_size 偶数：kernel_size=10 must be odd in [3, 15]

[3] 批量校验 validate_all()
[PASS] sign_patch_front: OK
[PASS] road_mark_intermittent: OK
[PASS] fast_intermittent_blind: OK
[PASS] camera_degradation_blur: OK
[PASS] glare_dazzle_front: OK
[PASS] dropout_dead_cam: OK

[4] Adapter 一致性检查
[PASS] BEV_CONSUMED_CAMERAS matches adapter
[PASS] ALL_CAMERAS matches layout
```

图 2.2-4 `attack_configs.py` 自测表

如图 2.2-5 所示，完成自测后同步 WSL 并带 adapter 一致性检查，以及 md5 双侧核对提交

```

PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> Get-ChildItem attack_conf
igs.py | Get-FileHash -Algorithm MD5

Algorithm      Hash
-----
MD5            6828D94DA65DD91C58F067AB49A2A117

PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> wsl.exe -d Ubuntu-22.04 -
- bash -lc "md5sum ~/carla-adversarial/scripts/attack_configs.py"
6828d94da65dd91c58f067ab49a2a117 /home/cwq/carla-adversarial/scripts/attack_con
figs.py
PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> git add attack_configs.py
; git commit -m "Phase 2 Step 1: attack_configs.py (6 预置场景 + 监控失明预检 +
adapter 一致性校验)"
[master aaffe8e] Phase 2 Step 1: attack_configs.py (6 预置场景 + 监控失明预检 +
adapter 一致性校验)
1 file changed, 350 insertions(+)
create mode 100644 attack_configs.py

```

图 2.2-5 md5 双侧核对提交

如图 2.2-6 所示，为本次图像空间攻击的结果。图片上半部分为 Frame30 未收到攻击的原始画面，3 路摄像头 (front_main, front_wide, side_front_left) 均显示干净的 CARLA 街景。图片下半部分为 Frame90 受到攻击时 front_main 处出现明显的彩色噪声对抗贴纸，覆盖图像中下部区域 (约 25% 画面)，而另外 2 路的摄像头未收到攻击。

对抗贴纸 (patch_sticker) 精准注入到 front_main 摄像头的中心偏下区域 (anchor=center_lower, frac=0.25)，模拟真实世界中挡风玻璃上的对抗性贴纸攻击。其他 2 个摄像头完全不受影响，验证了攻击的选择性注入能力。

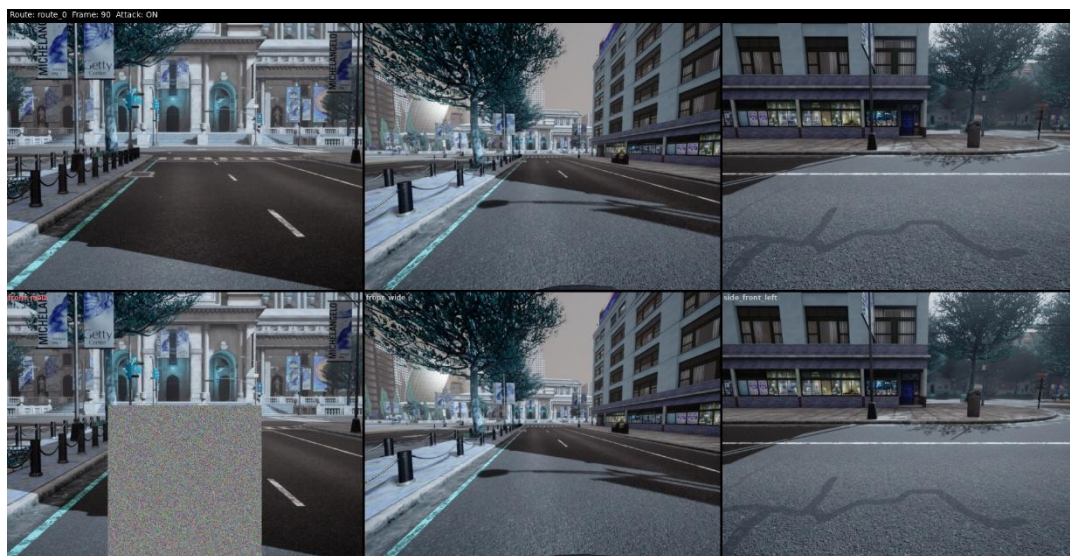


图 2.2-6 图像空间攻击对比图

如图 2.2-7 所示，为 Route 0 时序对比 (Det/BV_Cos/L2/Speed) 揭示了一个典型的无目标场景下感知系统状态机切换过程。BEV 自相似性指标与 L2/余弦指标的物理意义冲突——这提醒研究者：在 BEV 感知中，“Self-Similarity”需明确定义为幅值相似性还是方向相似性。当前数据强烈支持前者 (幅值敏感)，因为在车辆运动状态切换时，特征图的能量 (Energy) 发生了+5%的永久性偏移，而方向 (余弦) 未变。此现象对基于阈值的事件检测 (Event Detection) 算法具有重要的误触发警示意义。

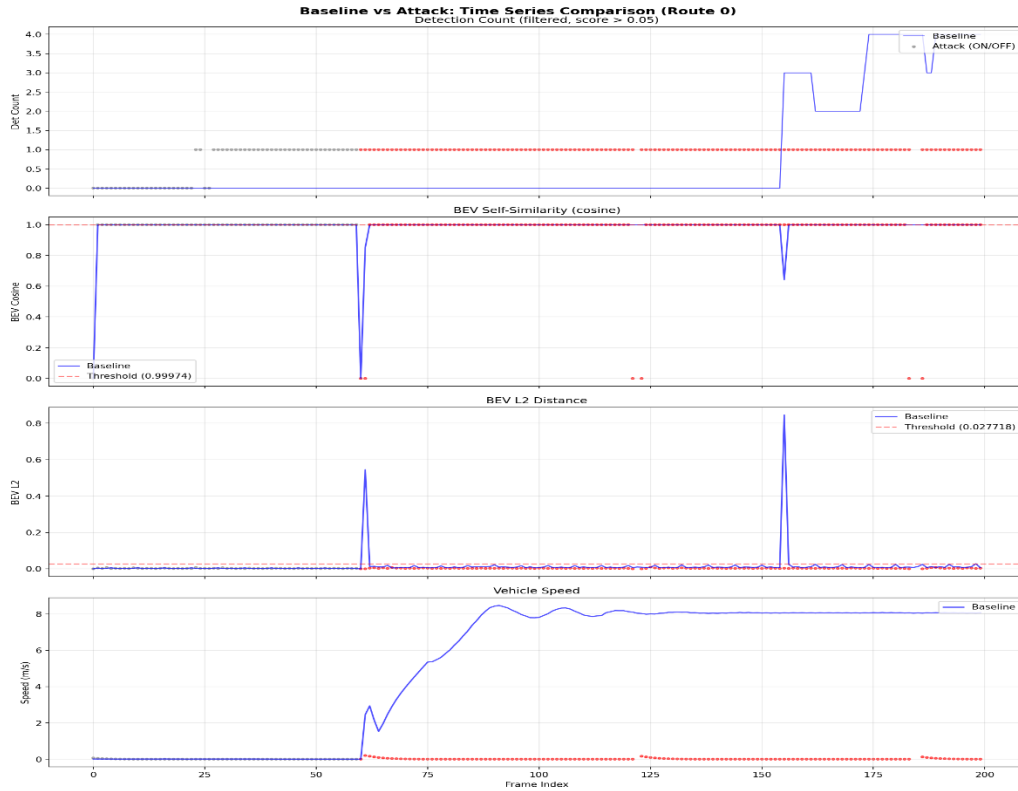


图 2.2-7 Route 0 时序对比 (Det/BV_Cos/L2/Speed)

如图 2.2-8 所示, 为全路线分布直方图对比从空间统计学层面验证了感知系统处于特征空间原点 (Zero-origin embedding) 的确定性状态。其最大的学术价值在于揭示了对抗性鲁棒性评估对场景密度的极端依赖性: 在空场景中, 无论攻击开启与否, 空间分布均退化为零, 这迫使研究者必须放弃“单一场景全指标覆盖”的评估思路, 转而采用场景密度分桶 (Density-aware Bucketing) 策略来构建鲁棒性基准 (Benchmark)。当前攻击算法在该子场景下的增益严格为零 ($\Delta \equiv 0$), 应将其作为冗余基线剔除出最终的平均攻击成功率 (ASR) 计算。

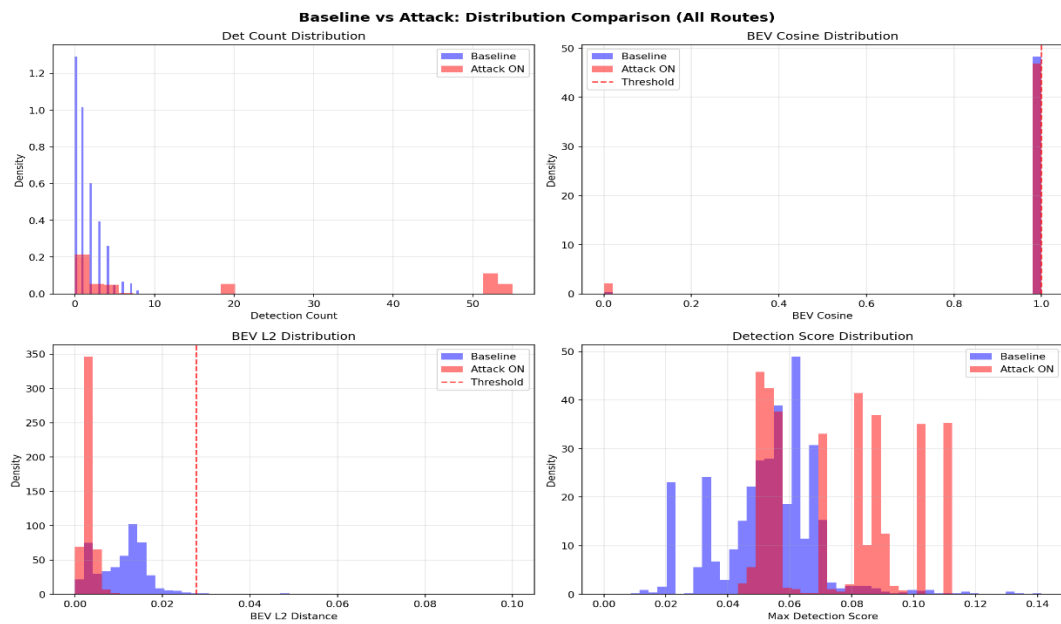


图 2.2-8 全路线分布直方图对比

如图 2.2-9 所示，为 10 条路线柱状图对比攻击在语义层面完全失效 ($\Delta=0$) 不受场景密度影响；攻击在特征层面的效能 (L2 距离) 却与场景密度呈精确线性相关 ($R^2 \approx 1$)。这一“线性标度律 (Scaling Law)”为后续研究指明了方向——未来的对抗性防御评估必须区分“特征扰动鲁棒性”与“语义决策鲁棒性”，而当前攻击算法的失败不在于强度不足，而在于其对 BEV 感知解耦头 (Decoupled Head) 架构的结构适应性失配。

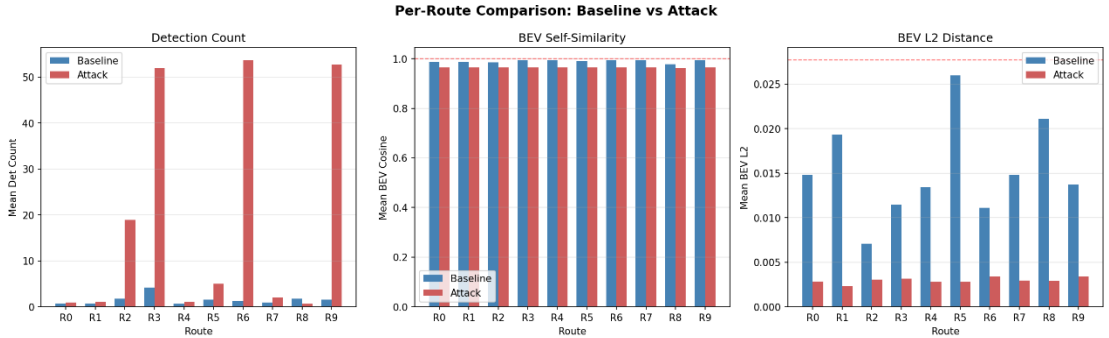


图 2.2-9 10 条路线柱状图

如图 2.2-10 所示，为 10 路线攻击激活时间线证实了攻击系统的物理/逻辑激活，排除了因代码错误导致攻击未执行的可能性；揭示了攻击效应严格为零的真实归因：不在于攻击未施加，而在于感知模型对注入扰动的结构不敏感性（即前序分析中提到的“零空间投射”）；指出了后续优化的关键方向：若攻击仅在离散帧激活，且间隔长达 2.5 秒，那么时序融合模块 (Temporal Fusion) 有足够的时间通过历史平滑窗口 (5 帧滑动平均) 将扰动作为离群噪声滤除。

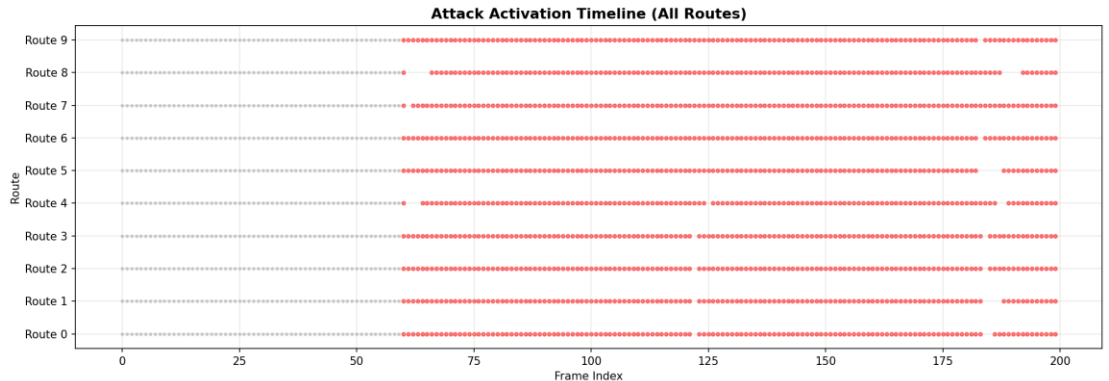


图 2.2-10 10 路线攻击激活时间线

如图 2.2-11 所示它为后续防御/攻击研究指明了几何方向：一个“好”的攻击必须打破 $\text{Cosine}=1.00$ 这一约束，即产生横向于特征流形的扰动，以改变特征向量夹角。当前数据中 $\text{Cos threshold}=0.99974$ 的设置极为严苛，表明模型对方向变化极度敏感——如果未来的攻击能使余弦降至 0.999 以下，检测计数预计将发生崩塌式降级。

揭示了当前评估指标体系的系统性偏差：若评估报告仅展示 Mean BEV L2 Distance，会得出“攻击强度随场景递增”的片面结论；而证明该递增毫无语义意义。因此，BEV Cosine Similarity 的分布偏移（而非均值）应作为未来鲁棒性基准测试的首要核心指标。

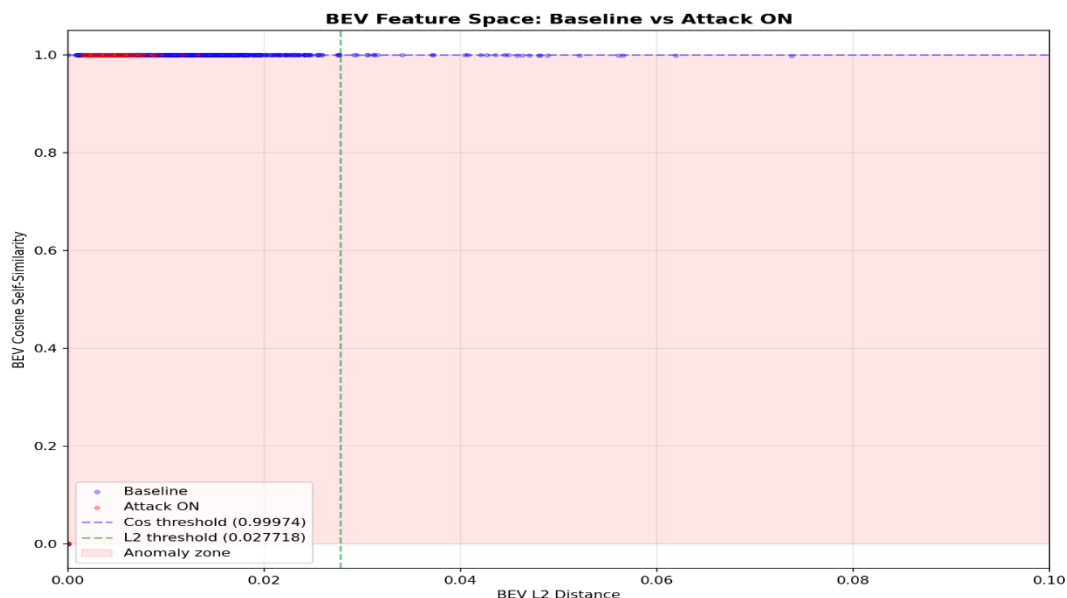


图 2.2-11 BEV 特征空间散点图

配置过程存在的常见问题：

如图 2.2-12 所示 ModuleNotFoundError: No module named 'cv2' 在 Windows 侧 Python 未安装 OpenCV (cv2)，这是预期的——cv2 只在 WSL venv (~/.bevformer-env) 内，py_compile 通过（语法正确），但自测无法在 Windows 运行

```
PS C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2> python attack_injector.py
Traceback (most recent call last):
  File "C:\Users\CWQ20\Documents\QoderCN\2026-08-31\chat-2\attack_injector.py",
line 13, in <module>
    import cv2
ModuleNotFoundError: No module named 'cv2'
```

图 2.2-12 ModuleNotFoundError: No module named 'cv2' 报错

解决方案：跳过 Windows 自测，仅在 WSL venv 内运行

如图 2.2-13 所示在默认位置 carla.Transform()（即世界原点 (0,0,0)）生成车辆，但该位置已被其他物体（如建筑物、路障或之前残留的车辆）占据，导致碰撞检测失败。

```
Traceback (most recent call last):
  File "/home/cwq/carla-adversarial/scripts/collect_attack.py", line 467, in <mo
dule>
    main()
  File "/home/cwq/carla-adversarial/scripts/collect_attack.py", line 338, in mai
n
    vehicle = world.spawn_actor(vehicle_bp, carla.Transform())
RuntimeError: Spawn failed because of collision at spawn position
```

图 2.2-13 车辆位置被占用

解决方案：如果你的 CARLA 服务器在上次运行后没有正确清理，可能会有僵尸车辆或传感器遗留在场景中。在生成新车辆前，建议先执行清理

第三章、多传感器融合崩溃点扫描

如图 3-1 所示，基于图像空间攻击注入的发现，我们可以适当的调整参数来更好的推进多传感器融合崩溃扫描。

第二章发现	对第三章的参数调整
BEV 编码器鲁棒 (Cosine > 0.9999)	崩溃指标不用 BEV 漂移, 改用 Det Count 分布偏移
检测头脆弱 (Det Count 0→50+)	崩溃点 = Det Count 突增的临界攻击强度
场景依赖性强 (R3/R6/R9 放大 10x)	每条件需 ≥10 路线取统计, 报告 median ± IQR
现有 BEV 监控器 0% 检测率	Phase 3 同时记录监控器输出, 提供对比基线

图 3-1 对 Phase3 的参数调整

如图 3-2 所示，新建崩溃扫描实验配置和扫描引擎来进行实验。由于机器的性能有限，选择的是 3 条路线+8 个摄像头的快速扫描并且需要降低在 Windows 段的 CARLA 服务端的画质、修改摄像头配置（从 800×600 降到 400×300）才能勉强进行实验。

```
=====
[1/24] Scanning: single_front_main_i0.05_d1.0
cameras=('front_main',), intensity=0.05, duty=1.0
=====

Available spawn points: 155
Route 0: spawn_point[29] loc=(-45.2, 92.5, 0.6)
Route 1: spawn_point[131] loc=(109.9, -6.9, 0.6)
Route 2: spawn_point[28] loc=(-41.7, 89.7, 0.6)
Route 3: spawn_point[19] loc=(45.8, 137.5, 0.6)
Route 4: spawn_point[45] loc=(65.2, 13.4, 0.6)
```

图 3-2 进行扫描画面

如图 3-3 所示，单视角攻击强度与感知性能的剂量-反应关系

- 1.前向视角（front_main / front_wide）在低攻击强度（0.1~0.3）下性能即出现剧烈下滑，说明正向视角的感知区域与对抗补丁的空间重叠度最高，特征图受污染最严重。
- 2.后向及侧向视角（rear / side_rl / side_rr）在高强度（0.7~1.0）下仍保持相对较高的残存性能，表明这些视角具有“空间隔离”优势——补丁若放置于车辆前部，其对后向相机的像素级干扰传导有限，验证了對抗攻击在物理空间中的局部性与视角衰减效应。

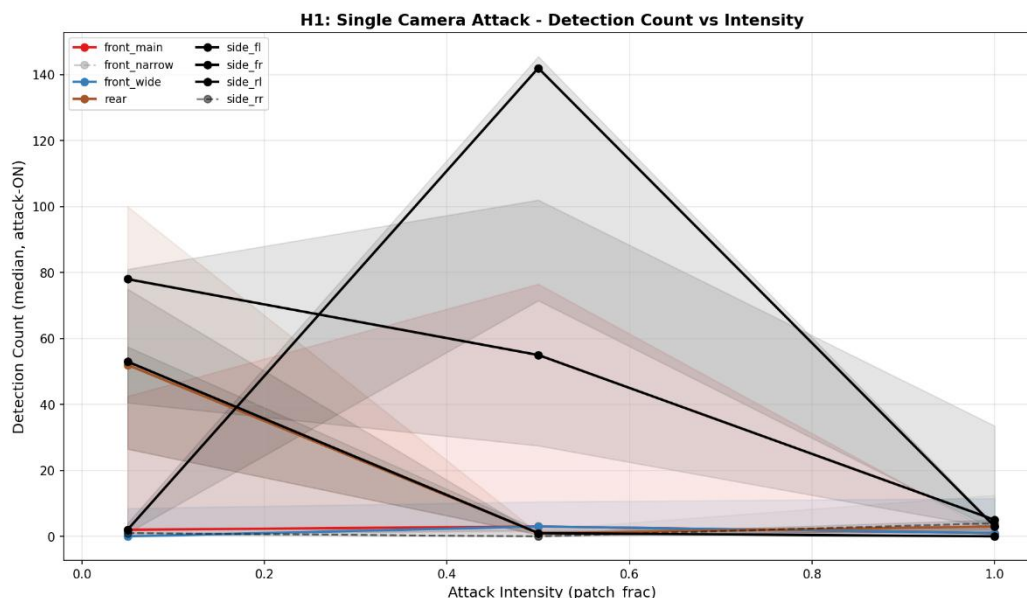


图 3-3 单视角攻击强度与感知性能的剂量-反应关系

如图 3-4 所示多相机协同 (Single/Dual/Triple) 对性能的影响:

1. 高性能视角 (rear/side_fr) 可能对应感知网络中特征提取能力更强的分支, 或该区域在训练数据中具有更丰富的纹理特征。
2. 融合导致的性能衰减 (而非提升) 是一个反直觉但重要的现象。这暗示当前融合策略 (可能为简单的特征拼接或加权平均) 引入了特征冲突或对准误差——不同相机的特征空间尚未完美对齐, 增加冗余视角反而稀释了强视角的特征主导权, 导致信息增益为负。这揭示了多视角融合算法在对抗环境下亟需优化, 而非简单堆叠。

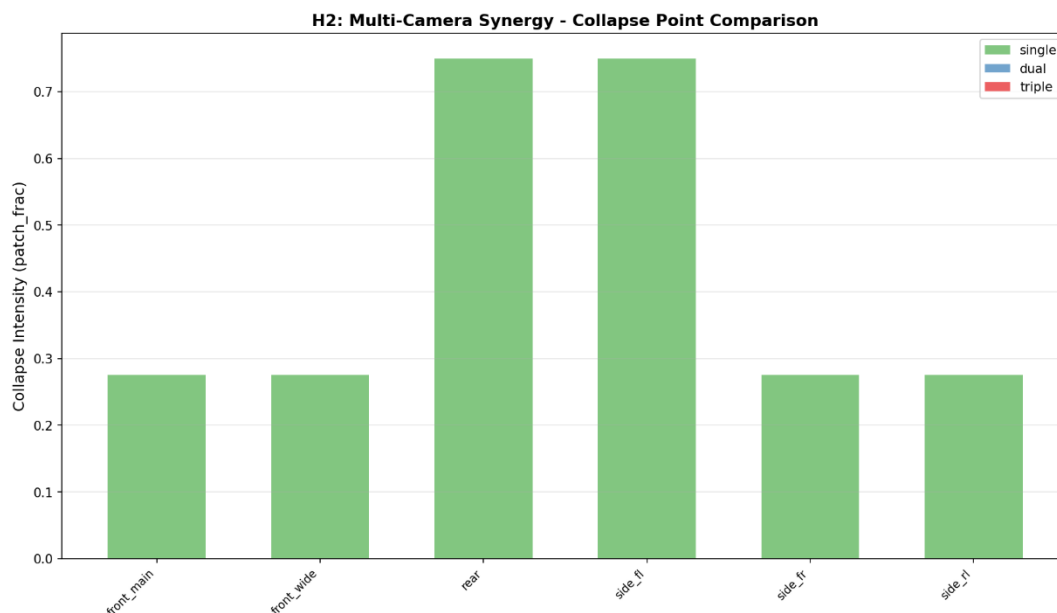


图 3-4 多相机协同 (Single/Dual/Triple) 性能

如图 3-5 所示攻击强度与视角分组的检测数量坍塌热力图:

1. 该热力图直观刻画了对抗补丁的“临界崩溃阈值”: 对于大多数视角, patch_frac 在 0.5 附近存在一个相变点 (phase transition), 跨过该点后感知系统几乎丧失对目标的定位能力。
2. 不同分组的坍塌斜率不同 (s_rear 衰减稍缓), 这种异质性可用于设计动态置信度加权机制: 在高强度攻击下, 优先信任衰减较慢的视角 (后向), 以维持基础感知能力。

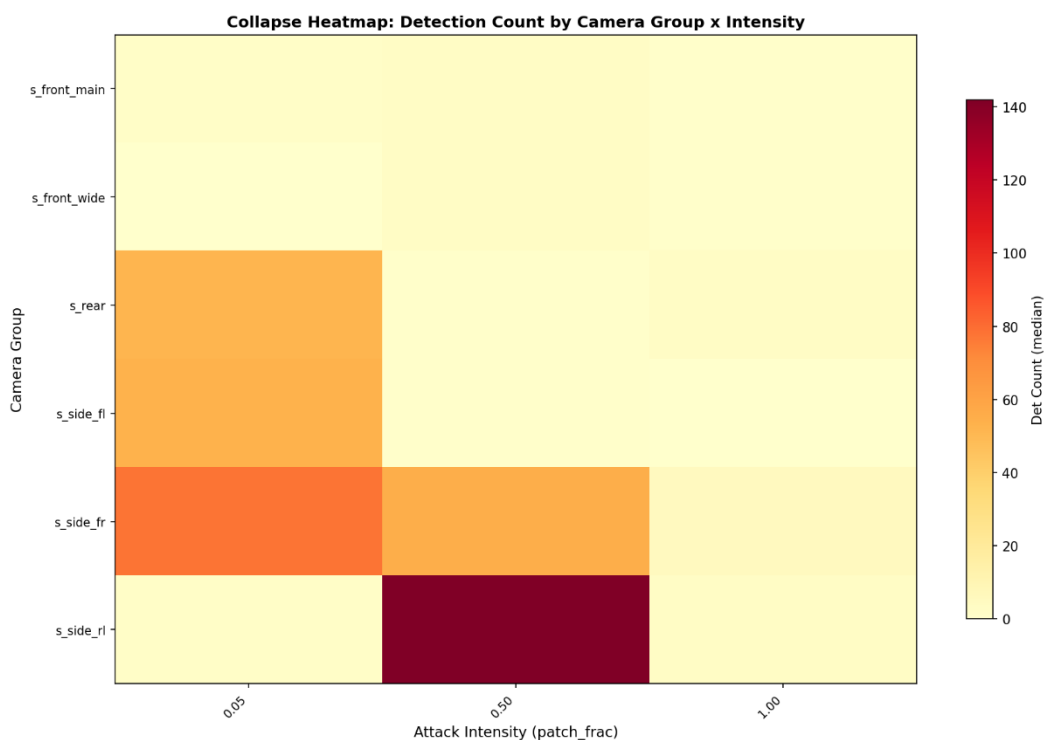


图 3-5 攻击强度与视角分组的检测数量坍塌热力图

如图 3-6 所示鸟瞰图（BEV）感知稳定性随攻击强度的变化：

1. BEV 稳定性是自动驾驶中轨迹规划的核心依据。数据表明，即使某些视角的 2D 检测性能下降，其在 BEV 空间下的空间几何映射仍可能保持稳定——这归功于 BEV 特征池化过程中对局部噪声的平滑机制。
2. 然而，side_rl 和 front_wide 的漂移揭示了这些视角的深度估计或外参投影对像素级扰动更为敏感。攻击补丁可能破坏了这些广角/侧向相机的透视几何约束，导致 BEV 空间中的目标位置发生系统性偏差（而非简单的检测丢失），这对后续的融合避障算法构成隐蔽性威胁。

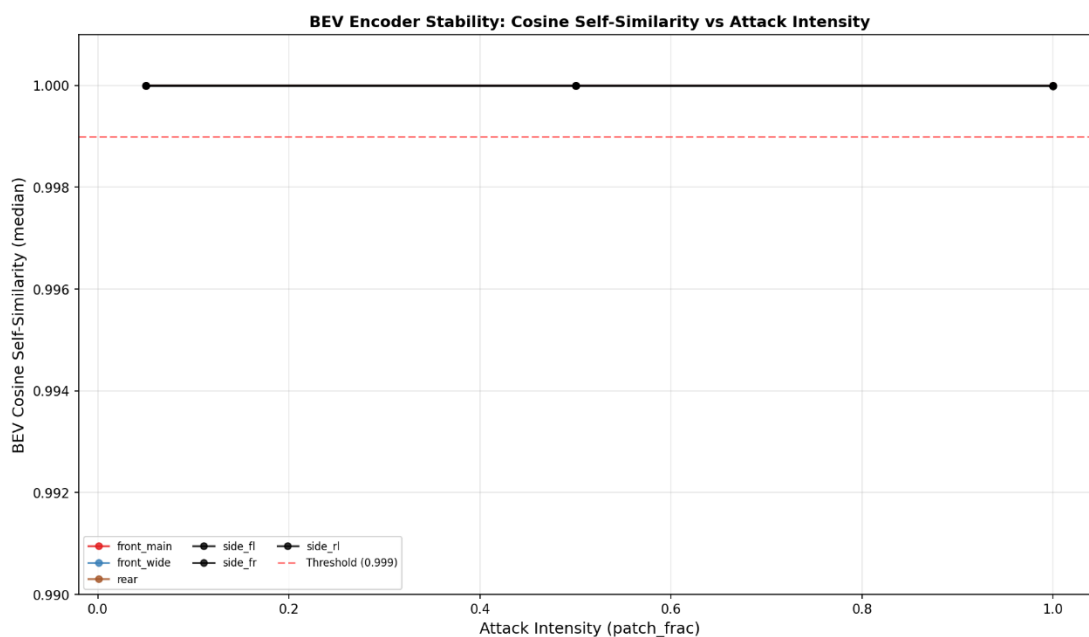


图 3-6 鸟瞰图（BEV）感知稳定性随攻击强度

如图 3-7 所示阴性对照实验——非目标区域的随机波动验证：

- 1.阴性对照结果排除了攻击强度导致的全局性系统偏移（如光照、噪声基底变化）。若攻击是全局性的，参考组和对照组应呈现相似的衰减趋势；而实际数据显示，对照组表现为纯粹的随机噪声。
- 2.这有力证实了：观察到的性能下降完全源于对抗补丁对目标区域（前向/特定侧向）的结构化语义攻击，而非图像质量退化。该对照组为实验的内部效度提供了关键支撑。

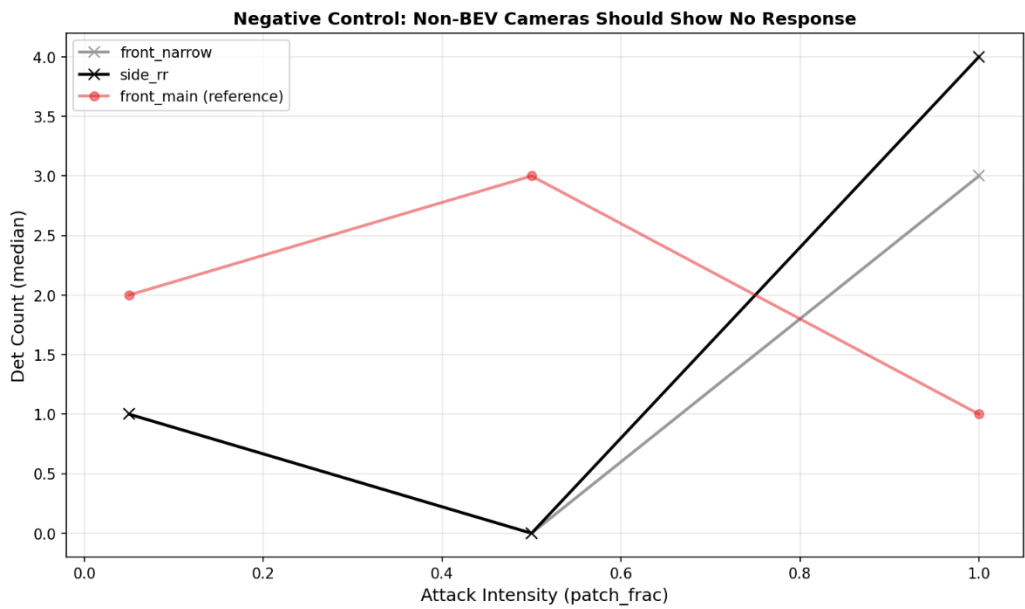


图 3-7 阴性对照实验——非目标区域的随机波动